

CI Expert – Residência em Microeletrônica
Trilha: RTL Design
Módulo: Projeto de Controlador Digital

Controlador de Vending Machine em SystemVerilog

Relatório Técnico

Aluno: Luciano Antunes de Oliveira

Projeto: Controlador digital de vending machine

Repositório GitHub: https://github.com/luciano066/grupo_NN_vending

Sumário

1	Introdução	2
2	Descrição Geral do Sistema	2
3	Metodologia de Desenvolvimento	2
4	Diagrama de Blocos	3
5	Máquina de Estados	4
6	Descrição dos Módulos	5
6.1	vending_pkg.sv	5
6.2	credit_reg.sv	5
6.3	memory.sv	5
6.4	comparator.sv	5
6.5	subtractor.sv	5
6.6	control_unit.sv	5
6.7	vending_top.sv	6
7	Simulação	6
8	Waveforms	6
8.1	Cenário 1: compra com troco	6
8.2	Cenário 2: crédito insuficiente	7
8.3	Cenário 3: cancelamento	7
8.4	Cenário 4: estoque zerado	8
9	Síntese	8
10	Exploração de Clock	9
11	Caminho Crítico	9
12	Comparação com e sem -no_autoungroup	10
13	Possíveis Melhorias Futuras	10
14	Conclusão	11

1 Introdução

Este relatório apresenta o desenvolvimento de um controlador digital de uma vending machine utilizando SystemVerilog. O projeto foi desenvolvido seguindo uma abordagem de projeto RTL, com separação entre unidade de controle e caminho de dados. A arquitetura integra uma máquina de estados finitos, memória síncrona, registrador de crédito, comparador e subtrator.

A máquina implementada possui quatro produtos: café, água, suco e snack. O usuário insere moedas, seleciona um item, confirma a compra e recebe o produto e o troco quando a venda é válida. Em casos de crédito insuficiente ou estoque zerado, a máquina entra em estado de erro.

O projeto foi validado inicialmente por simulação funcional com Synopsys VCS, analisado por waveforms no Verdi e sintetizado com Synopsys Design Compiler utilizando a biblioteca SAED32. Além da validação funcional, foram analisados área, timing, caminho crítico e impacto de diferentes restrições de clock.

2 Descrição Geral do Sistema

A vending machine recebe como entradas principais o valor da moeda inserida, a seleção do item, o sinal de confirmação da compra e o sinal de cancelamento. As saídas principais indicam liberação de produto, troco, erro, crédito exibido e estado atual da FSM.

Os valores monetários foram representados internamente em centavos, usando registradores de 8 bits. Essa escolha simplifica as operações aritméticas, pois evita o uso de ponto flutuante e permite representar todos os valores necessários para a aplicação.

Os valores das moedas são codificados conforme a Tabela 1.

Tabela 1: Codificação das moedas

coin_in	Valor	Representação interna
00	R\$0,00	0
01	R\$0,25	25
10	R\$0,50	50
11	R\$1,00	100

Os produtos armazenados na memória são apresentados na Tabela 2.

Tabela 2: Produtos da vending machine

Endereço	Item	Preço	Preço interno	Estoque inicial
0	Café	R\$0,25	25	5
1	Água	R\$0,50	50	5
2	Suco	R\$0,75	75	3
3	Snack	R\$1,00	100	2

O comportamento esperado do sistema pode ser resumido da seguinte forma: a máquina inicia em repouso, acumula créditos quando moedas são inseridas, verifica se a compra é válida, libera o produto quando há crédito suficiente e estoque disponível, calcula o troco e retorna ao estado inicial. Caso a venda não possa ser realizada, a FSM entra em estado de erro.

3 Metodologia de Desenvolvimento

O desenvolvimento do controlador seguiu uma metodologia típica de projeto RTL para circuitos digitais síncronos. Inicialmente, o comportamento da vending machine foi decomposto em duas partes principais: a unidade de controle e o caminho de dados. Essa separação é uma prática comum em projetos digitais, pois permite isolar a lógica responsável pela sequência de operações da lógica responsável pelo armazenamento e processamento dos dados.

A unidade de controle foi implementada como uma máquina de estados finitos do tipo Moore, na qual as saídas dependem apenas do estado atual. Essa escolha facilita a análise temporal do circuito, pois reduz a dependência direta entre entradas externas e saídas de controle. O caminho de dados foi composto por módulos combinacionais e sequenciais responsáveis por acumular crédito, armazenar preço e estoque, comparar condições de venda e calcular o troco.

Após a implementação RTL em SystemVerilog, o projeto foi validado por simulação funcional utilizando um testbench self-checking. Esse tipo de testbench verifica automaticamente os resultados esperados, reduzindo a dependência de inspeção manual das waveforms. Em seguida, o circuito foi sintetizado com o Synopsys Design Compiler utilizando a biblioteca SAED32, permitindo obter métricas de área, timing, potência e identificar o caminho crítico.

O fluxo utilizado foi: definição da arquitetura, implementação RTL, criação do testbench self-checking, simulação funcional com VCS, análise das waveforms no Verdi, síntese lógica com Design Compiler, análise dos relatórios de área, timing, potência e constraints, e exploração do período de clock para estimar a frequência máxima.

4 Diagrama de Blocos

A Figura 1 apresenta o diagrama de blocos simplificado do sistema. O módulo `vending_top` funciona como o bloco integrador do projeto, conectando a unidade de controle ao caminho de dados.

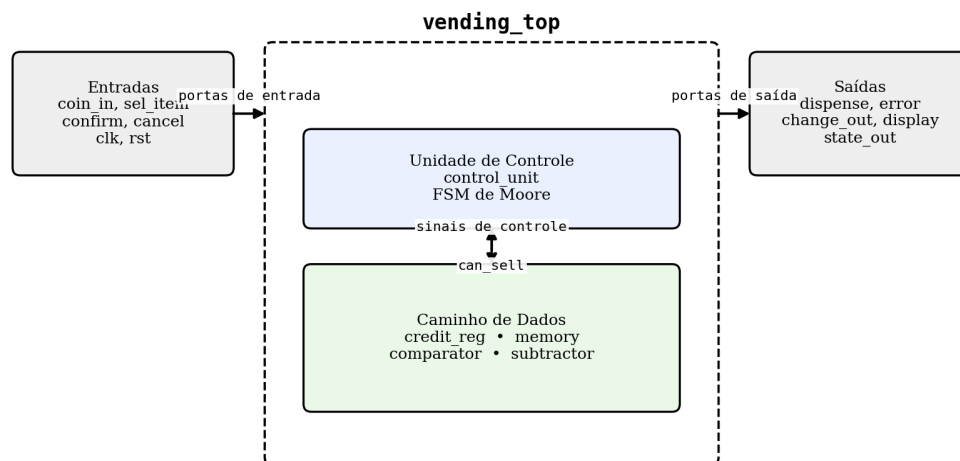


Diagrama propositalmente simplificado: mostra arquitetura, não todos os fios.

Figura 1: Diagrama de blocos simplificado do controlador da vending machine.

O diagrama evidencia a separação entre a unidade de controle e o caminho de dados. A unidade de controle é responsável pela sequência dos estados e pela geração dos sinais de controle. O caminho de dados concentra os módulos responsáveis por armazenar crédito, acessar preço e estoque, verificar se a venda pode ocorrer e calcular o troco.

Do ponto de vista arquitetural, o projeto segue o modelo clássico de separação entre controle e datapath. A unidade de controle não realiza operações aritméticas diretamente; sua função é decidir quando cada operação deve ocorrer. Já o caminho de dados contém os elementos que armazenam e processam informações, como o crédito acumulado, o preço do item selecionado, o estoque disponível e o valor do troco.

Essa separação facilita a manutenção do projeto e também torna a síntese mais previsível. Caso seja necessário alterar a política de funcionamento da máquina, como adicionar novos estados ou modificar a sequência de venda, a maior parte das mudanças ocorre na FSM. Caso seja necessário alterar os dados processados, como aumentar o número de produtos ou mudar a largura dos valores monetários, as mudanças se concentram no caminho de dados.

5 Máquina de Estados

A unidade de controle foi implementada como uma FSM de Moore com seis estados: IDLE, COLLECT, CHECK, DISPENSE, CHANGE e ERROR. As saídas dependem apenas do estado atual.

Tabela 3: Estados da FSM

Estado	Codificação	Função
IDLE	000	Espera inserção de moeda
COLLECT	001	Acumula crédito
CHECK	010	Verifica crédito e estoque
DISPENSE	011	Libera produto e decrementa estoque
CHANGE	100	Registra troco e zera crédito
ERROR	101	Indica erro de venda

A escolha por uma FSM de Moore foi feita porque, nesse modelo, as saídas são determinadas pelo estado atual e não diretamente pelas entradas. Isso reduz a possibilidade de glitches nas saídas de controle e torna o comportamento mais previsível em um circuito síncrono. Como a máquina é controlada por clock, as transições de estado ocorrem na borda de subida de `clk`, garantindo que o sistema avance de forma ordenada.

O estado **CHECK** é especialmente importante porque nele a FSM avalia se a venda pode ocorrer. Essa decisão depende de dois fatores: crédito suficiente e estoque disponível. Esses critérios são sintetizados no sinal `can_sell`, produzido pelo comparador. Quando `can_sell = 1`, a máquina libera o produto no estado **DISPENSE**. Quando `can_sell = 0`, a máquina entra no estado **ERROR**.

A Figura 2 apresenta o fluxo principal da FSM. Em uma compra válida, o fluxo é **IDLE** → **COLLECT** → **CHECK** → **DISPENSE** → **CHANGE** → **IDLE**. Caso a venda não possa ser realizada, a FSM vai para **ERROR**.

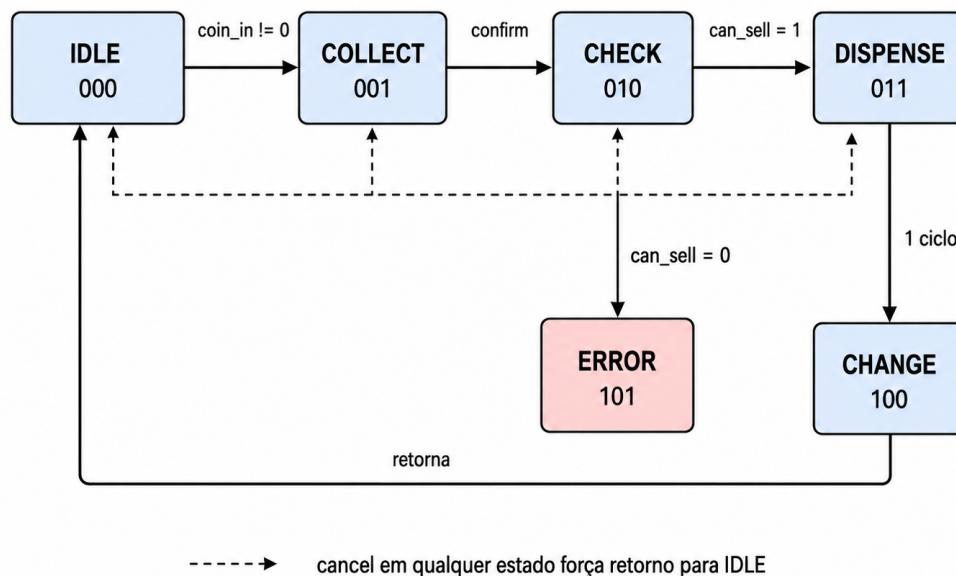


Figura 2: Diagrama de transição de estados da FSM.

O caminho normal de uma compra bem-sucedida é composto por cinco etapas: espera por moeda, acúmulo de crédito, verificação da venda, liberação do produto e devolução de troco. Esse fluxo

corresponde à sequência `IDLE → COLLECT → CHECK → DISPENSE → CHANGE → IDLE`. Já situações inválidas, como crédito insuficiente ou estoque zerado, desviam o fluxo para `ERROR`.

O sinal `cancel` possui prioridade e força o retorno ao estado `IDLE`, zerando o crédito e registrando o valor a ser devolvido ao usuário.

6 Descrição dos Módulos

6.1 `vending_pkg.sv`

Este arquivo contém o pacote `vending_pkg`, utilizado para centralizar a definição dos estados da FSM. A utilização de um `typedef enum logic [2:0]` melhora a legibilidade do projeto e evita o uso direto de valores numéricos ao longo do código. Centralizar os estados em um pacote também facilita a reutilização das definições por diferentes módulos do projeto.

6.2 `credit_reg.sv`

O módulo `credit_reg` implementa o registrador de crédito acumulado da máquina. Ele é um bloco sequencial sensível à borda de subida do clock e ao sinal de reset. Internamente, o módulo converte a codificação binária da moeda de entrada para um valor em centavos por meio da função `coin_to_value`. Assim, `coin_in = 01` representa 25 centavos, `coin_in = 10` representa 50 centavos e `coin_in = 11` representa 100 centavos.

A lógica do registrador possui uma ordem de prioridade. O reset possui prioridade máxima, pois inicializa o crédito em zero. Em seguida, o cancelamento devolve o crédito acumulado e zera o registrador. O sinal `clear_credit` também zera o crédito após uma compra concluída. Quando nenhuma dessas condições está ativa, o registrador acumula o valor da moeda inserida.

6.3 `memory.sv`

O módulo `memory` representa a memória interna da vending machine. Ele armazena preço e estoque dos quatro produtos disponíveis. Cada posição possui 16 bits, sendo os 8 bits mais significativos utilizados para armazenar o preço e os 8 bits menos significativos utilizados para armazenar o estoque.

A leitura da memória é síncrona, ou seja, os valores de preço e estoque são atualizados em uma borda de clock. A escrita também é síncrona e ocorre quando a FSM libera a venda, decrementando o estoque do produto selecionado. Dessa forma, o módulo modela tanto a consulta ao produto quanto a atualização do estoque após uma compra.

6.4 `comparator.sv`

O comparador é uma lógica combinacional que gera o sinal `can_sell`. Esse sinal é verdadeiro quando duas condições são satisfeitas simultaneamente: o crédito acumulado é maior ou igual ao preço do item e o estoque do item é maior que zero.

Esse módulo concentra a condição de validade da venda em um único sinal. Assim, a unidade de controle não precisa conhecer os detalhes da comparação entre crédito, preço e estoque; ela apenas utiliza `can_sell` para decidir a transição entre os estados `DISPENSE` e `ERROR`.

6.5 `subtractor.sv`

O subtrator calcula o troco por meio da operação `credit - price`. Esse resultado é usado quando a FSM entra no estado `CHANGE`. Como a venda só ocorre após `can_sell` ser verdadeiro, a subtração não gera valor negativo durante uma compra válida.

A separação dessa operação em um módulo próprio melhora a organização do datapath e facilita a análise do circuito.

6.6 `control_unit.sv`

A `control_unit` é o núcleo de controle do projeto. Ela implementa uma FSM responsável por coordenar a operação dos demais módulos. A partir das entradas externas e do sinal `can_sell`, a

FSM decide quando acumular crédito, quando consultar a memória, quando liberar o produto, quando calcular o troco e quando indicar erro.

Um ponto relevante do projeto é o uso do sinal `check_valid`. Como a memória possui leitura síncrona, o preço e o estoque não ficam disponíveis imediatamente no mesmo ciclo em que o endereço é selecionado. O sinal `check_valid` ajuda a garantir que a FSM tome a decisão de venda apenas quando os dados lidos da memória já estão válidos.

6.7 vending_top.sv

O módulo `vending_top` é o módulo de integração do projeto. Ele conecta a unidade de controle ao caminho de dados e expõe a interface externa da vending machine. Esse módulo instancia a FSM, o registrador de crédito, a memória, o comparador e o subtrator.

Além de realizar a interconexão entre os blocos, o `vending_top` também registra algumas saídas, como `change_out` e `display`. Registrar saídas é uma prática importante em projetos síncronos, pois contribui para um comportamento temporal mais estável e facilita a análise de timing durante a síntese.

7 Simulação

A simulação foi realizada com Synopsys VCS utilizando um testbench self-checking. O testbench implementa tarefas para aplicar moedas, confirmar compras, cancelar operações, esperar estados específicos e verificar automaticamente os resultados esperados.

Um testbench self-checking é mais robusto que uma simulação baseada apenas em inspeção visual, pois ele compara automaticamente os valores observados com os valores esperados. Dessa forma, falhas funcionais podem ser detectadas diretamente pelo terminal, por meio de mensagens de `PASS` e `FAIL`.

Os quatro cenários obrigatórios foram verificados: compra bem-sucedida com troco, crédito insuficiente, cancelamento e estoque zerado.

A saída final do testbench foi:

```
PASS = 23
FAIL = 0
TODOS OS TESTES PASSARAM.
```

Esse resultado indica que todos os cenários funcionais planejados foram executados corretamente.

8 Waveforms

As waveforms foram analisadas no Verdi. Como o DVE não estava disponível no ambiente utilizado, as waveforms foram analisadas no Verdi, ferramenta também pertencente ao fluxo Synopsys. Os principais sinais observados foram: `clk`, `rst`, `coin_in`, `sel_item`, `confirm`, `cancel`, `state_out`, `dispense`, `error`, `change_out`, `display`, `credit`, `price` e `stock`.

A análise das waveforms complementa o testbench self-checking porque permite verificar visualmente a sequência temporal dos sinais. Essa etapa é importante para confirmar que a FSM percorre os estados esperados e que os sinais de controle são ativados nos ciclos corretos.

8.1 Cenário 1: compra com troco

Neste cenário, foi inserida uma moeda de R\$1,00 para comprar café, cujo preço é R\$0,25. O resultado esperado era liberação do produto e troco de 75 centavos.

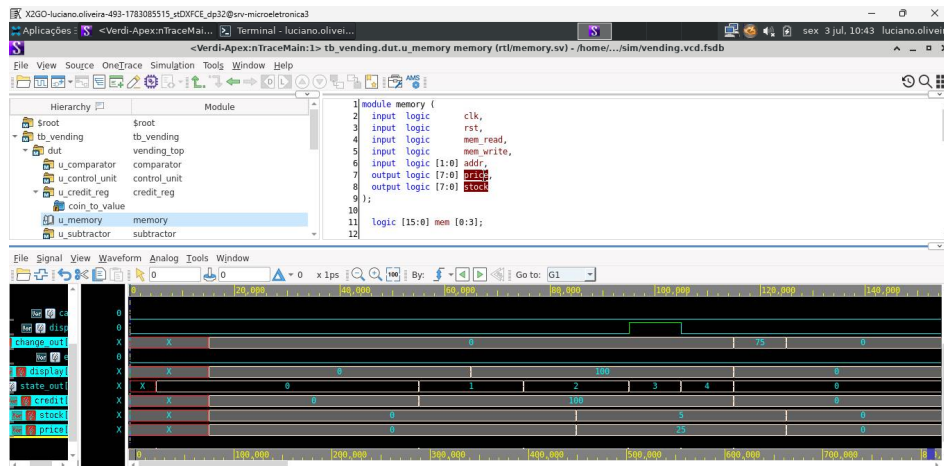


Figura 3: Waveform do cenário 1: compra bem-sucedida com troco.

Na waveform, observa-se o crédito acumulado de 100 centavos, o preço de 25 centavos, a ativação de **dispense** e o valor de troco igual a 75 centavos em **change_out**. Isso confirma que a operação ocorreu corretamente.

8.2 Cenário 2: crédito insuficiente

Neste cenário, foi inserido crédito de 25 centavos para tentar comprar o snack, cujo preço é 100 centavos. Como o crédito acumulado é menor que o preço do item, a FSM deve ir para o estado **ERROR**.

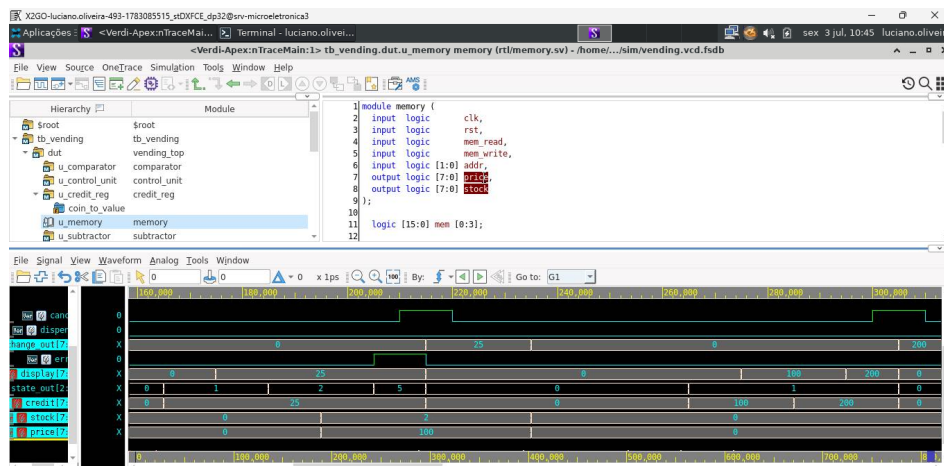


Figura 4: Waveform do cenário 2: crédito insuficiente.

A waveform mostra que, ao avaliar a condição de venda, o sistema identifica que **can_sell** não deve permitir a compra. Como consequência, a FSM entra no estado de erro, indicado por **state_out** = 5 e pela ativação do sinal **error**.

8.3 Cenário 3: cancelamento

Neste cenário, foram inseridas duas moedas de R\$1,00 e em seguida o usuário acionou **cancel**. O crédito foi zerado, a FSM retornou para **IDLE** e **change_out** recebeu 200 centavos.

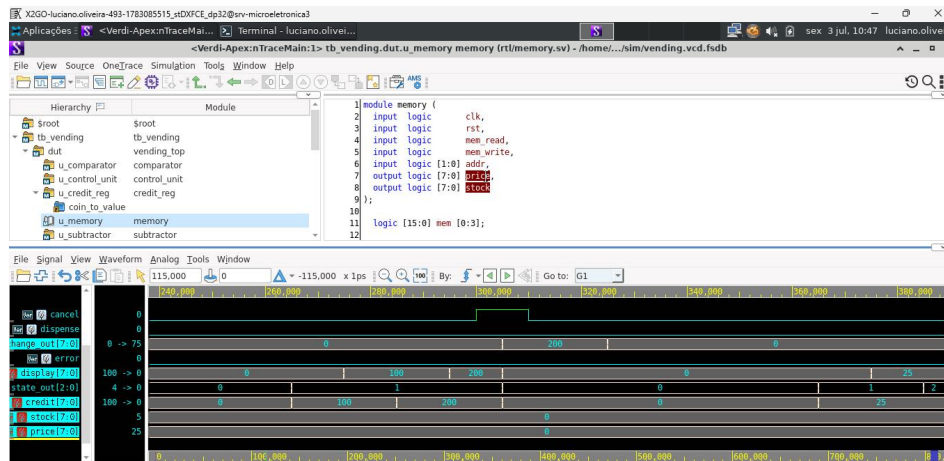


Figura 5: Waveform do cenário 3: cancelamento.

A waveform confirma que o crédito acumulado chegou a 200 centavos e, após a ativação de `cancel`, esse valor foi transferido para `change_out`. Em seguida, o crédito foi zerado e a FSM retornou para o estado IDLE.

8.4 Cenário 4: estoque zerado

Neste cenário, o café foi comprado cinco vezes até esgotar o estoque. Na sexta tentativa, o estoque estava em zero e a FSM foi para ERROR.

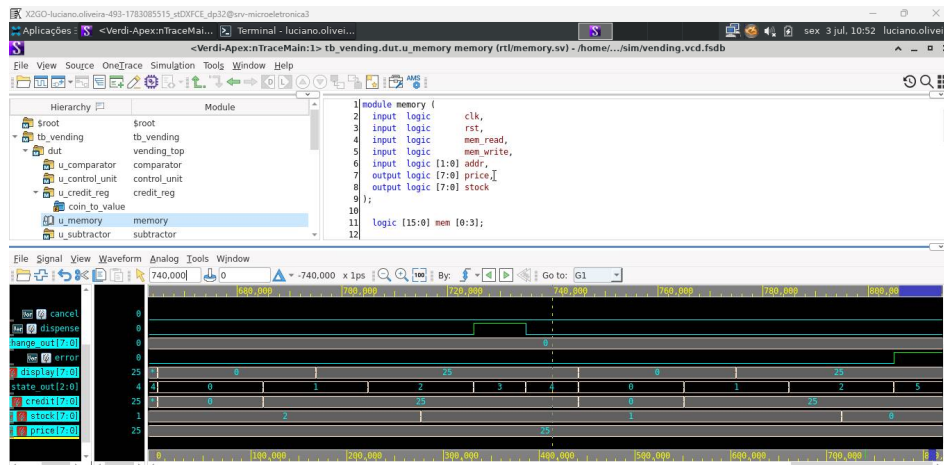


Figura 6: Waveform do cenário 4: estoque zerado.

A waveform mostra o estoque chegando a zero e, na tentativa seguinte de compra, a FSM entrando no estado ERROR. Esse comportamento confirma que a lógica de estoque está sendo considerada corretamente pelo comparador e pela unidade de controle.

9 Síntese

A síntese foi realizada com Synopsys Design Compiler utilizando a biblioteca SAED32 `saed32rvt_tt1p05v25c.d`. O arquivo de constraints inicial definiu clock de 20 ns, equivalente a 50 MHz.

A síntese lógica transforma a descrição RTL em uma netlist composta por células da biblioteca tecnológica. Nessa etapa, o Design Compiler realiza mapeamento lógico, otimizações de área e timing, além de gerar relatórios que permitem avaliar a qualidade do circuito resultante.

Os resultados iniciais são apresentados na Tabela 4.

Tabela 4: Resultado inicial de síntese

Métrica	Valor
Período de clock	20 ns
Frequência	50 MHz
Área total	1122.808214
Slack	14.56 ns

O slack positivo indica que, para o período inicial de 20 ns, o circuito atende confortavelmente à restrição temporal. Como a margem é grande, o período de clock foi progressivamente reduzido para avaliar até que frequência o circuito poderia operar sem violar timing.

10 Exploração de Clock

Após a síntese inicial, o período do clock foi reduzido progressivamente para estimar a frequência máxima de operação do circuito. Os resultados são apresentados na Tabela 5.

Tabela 5: Exploração de período de clock

Período	Frequência	Slack	Área	Resultado
20 ns	50.00 MHz	14.56 ns	1122.808214	MET
18 ns	55.56 MHz	12.56 ns	1122.808214	MET
16 ns	62.50 MHz	10.56 ns	1122.808214	MET
14 ns	71.43 MHz	8.56 ns	1122.808214	MET
12 ns	83.33 MHz	6.56 ns	1122.808214	MET
10 ns	100.00 MHz	4.56 ns	1122.808214	MET
8 ns	125.00 MHz	2.56 ns	1122.808214	MET
6 ns	166.67 MHz	0.56 ns	1122.808214	MET
5 ns	200.00 MHz	0.06 ns	1129.670102	MET
4 ns	250.00 MHz	-0.19 ns	1462.344604	VIOLATED

A análise da Tabela 5 mostra que, para períodos maiores, o circuito apresenta grande folga temporal. Em 20 ns, o slack é de 14.56 ns, indicando que o caminho mais lento do circuito utiliza apenas uma fração do tempo disponível. À medida que o período de clock é reduzido, a frequência aumenta e a margem temporal diminui.

O circuito continuou atendendo às restrições até o período de 5 ns, correspondente a 200 MHz, com slack positivo de 0.06 ns. Ao reduzir o período para 4 ns, correspondente a 250 MHz, o slack se tornou negativo, indicando violação de timing. Portanto, a frequência máxima confirmada experimentalmente foi de 200 MHz.

Também é possível observar o aumento de área quando o período de clock se torna mais agressivo. Em períodos de 20 ns até 6 ns, a área permaneceu praticamente constante. Porém, em 5 ns e principalmente em 4 ns, a área aumentou, pois a ferramenta de síntese tenta usar células mais rápidas ou reorganizar caminhos para reduzir atrasos. Esse comportamento ilustra o compromisso entre área e desempenho.

11 Caminho Crítico

Para o período de 4 ns, o caminho crítico reportado pelo Design Compiler foi:

```
Startpoint: coin_in[1]
Endpoint: u_credit_reg/credit_reg[5]
Slack: -0.19 ns
```

Esse caminho atravessa o módulo `credit_reg`. Ele parte da entrada `coin_in[1]`, passa pela lógica de decodificação da moeda e pela lógica combinacional responsável pela atualização do acumulador de crédito, chegando ao flip-flop `credit_reg[5]`.

A violação observada em 4 ns é uma violação de setup. Em circuitos síncronos, o dado precisa chegar à entrada do flip-flop de destino antes da próxima borda ativa de clock, respeitando o tempo de setup da célula sequencial. Quando o tempo de chegada do dado é maior que o tempo requerido, o slack se torna negativo, indicando que o circuito pode não operar corretamente naquela frequência.

No caso deste projeto, o caminho crítico parte da entrada `coin_in[1]` e termina no registrador `u_credit_reg/credit_reg[5]`. Isso indica que a atualização do crédito acumulado é o trecho mais sensível do circuito. Esse caminho inclui a decodificação da moeda e a lógica combinacional de soma responsável por calcular o novo valor de crédito.

Portanto, a operação de acúmulo de crédito é o principal limitador de frequência do projeto em períodos de clock mais agressivos. Uma possível modificação no RTL para reduzir o caminho crítico seria registrar previamente o valor decodificado da moeda, separando a etapa de conversão de `coin_in` da etapa de soma com `credit`. Isso reduziria a profundidade combinacional antes do registrador de crédito, embora pudesse introduzir um ciclo adicional de latência.

12 Comparação com e sem `-no_autoungroup`

Também foi feita uma síntese em 5 ns removendo a opção `-no_autoungroup`. Os resultados são apresentados na Tabela 6.

Tabela 6: Comparação com e sem auto-ungroup

Configuração	Área	Slack
<code>compile_ultra -no_autoungroup</code>	1129.670102	0.06 ns
<code>compile_ultra</code>	1045.548436	0.44 ns

Ao remover a opção `-no_autoungroup`, o Design Compiler teve mais liberdade para otimizar a hierarquia do projeto. Isso resultou em menor área e maior slack, indicando uma otimização mais eficiente.

Esse resultado mostra que a preservação rígida da hierarquia nem sempre produz a melhor implementação lógica. Em alguns casos, permitir que a ferramenta desfaça limites hierárquicos internos possibilita otimizações entre módulos, reduzindo lógica redundante e melhorando caminhos críticos. Por outro lado, preservar a hierarquia pode facilitar depuração, leitura da netlist e rastreabilidade entre RTL e circuito sintetizado.

13 Possíveis Melhorias Futuras

Algumas melhorias poderiam ser realizadas em versões futuras do projeto. Uma primeira possibilidade seria parametrizar o número de produtos e a largura dos registradores, tornando o controlador mais flexível. Dessa forma, a vending machine poderia ser expandida para mais itens sem a necessidade de reescrever manualmente a memória e parte da lógica de seleção.

Outra melhoria seria registrar o valor decodificado da moeda antes da soma com o crédito. Essa alteração separaria a etapa de decodificação de `coin_in` da etapa de acúmulo em `credit`. Como o caminho crítico identificado envolve justamente a atualização do registrador de crédito, essa modificação poderia reduzir a profundidade combinacional e melhorar a frequência máxima de operação.

Também seria possível expandir o testbench para incluir testes aleatórios, verificações com assertions SystemVerilog e cobertura funcional. Do ponto de vista de síntese, uma análise mais completa poderia incluir diferentes cantos de processo, tensão e temperatura, além de comparação entre bibliotecas de células com diferentes características de desempenho e consumo.

14 Conclusão

O projeto do controlador de vending machine foi implementado com sucesso em SystemVerilog, seguindo uma organização típica de projeto RTL baseada na separação entre unidade de controle e caminho de dados. A unidade de controle foi implementada como uma FSM de Moore, enquanto o caminho de dados concentrou os módulos responsáveis pelo armazenamento e processamento de crédito, preço, estoque e troco.

A validação funcional foi realizada por meio de um testbench self-checking, que verificou automaticamente os principais cenários de funcionamento da máquina: compra com troco, crédito insuficiente, cancelamento e estoque zerado. O resultado final da simulação apresentou 23 verificações aprovadas e nenhuma falha, indicando que o comportamento funcional esperado foi atendido.

A síntese lógica foi realizada com o Synopsys Design Compiler utilizando a biblioteca SAED32. A análise de timing mostrou que o circuito atende às restrições até o período de 5 ns, correspondente a uma frequência de 200 MHz. Para o período de 4 ns, foi observada violação de timing com slack negativo de -0.19 ns. O caminho crítico foi identificado no módulo `credit_reg`, mais especificamente na lógica de atualização do crédito acumulado a partir da entrada `coin_in`.

A comparação entre as versões com e sem `-no_autoungroup` mostrou que permitir o auto-ungroup possibilitou melhor otimização pela ferramenta, reduzindo a área e melhorando o slack. Esse resultado evidencia a influência das opções de síntese sobre a qualidade final do circuito em termos de área e desempenho.

Como aprendizado, o projeto demonstrou a importância de uma boa decomposição arquitetural, da construção de testbenches automatizados, da definição adequada de constraints e da análise dos relatórios de síntese. Além disso, a identificação do caminho crítico permitiu compreender quais partes do RTL limitam a frequência máxima de operação e quais modificações poderiam ser consideradas em uma futura otimização.